DATABASE MANAGEMENT SYSTEMS

FINTRACK

Expense Tracker & Split Money

PROJECT IDEA SUBMISSION

PROJECT FOCUS

A relational database for recording income and expenses, managing friend and group relationships, allocating shared costs, and tracking repayments.

PROJECT TEAM

Team member	Student ID
Kartikey Kansal	2025UCS1640
Avani Gupta	2025UCS1637
Vihaan	2025UCS1621
Nishchay singh	2025UCS1646

Contents

Section	Title
01	Problem Statement
02	Objectives
03	Project Scope and Requirements
04	Project Modules
05	Existing Similar Projects
06	Gaps and Proposed Novelty
07	Schema Diagram
08	ER Diagram
09	Relational Data Dictionary
10	Conclusion
11	References

1. Problem Statement

FinTrack is a proposed expense-tracking system designed to make managing shared money easier. It helps friends, roommates, and travel groups keep track of their expenses, split bills, and see who owes whom. The system uses a relational database to store and organize expense records, making it easier to manage shared costs and keep financial information organized.

Problem statement

- When expenses are recorded across chats, notes, or spreadsheets, information can get scattered and mistakes can easily happen. It can also be difficult to track repayments or understand how a particular balance was calculated.
- FinTrack aims to address these issues through a relational database that connects users, groups, transactions, expense shares, and settlements. This helps keep records consistent and makes it easier to trace outstanding dues back to their original transactions.

2. Objectives

- Manage user profiles and allow multiple login providers to be linked to one account.
- Track friendships, including requests, status changes, and the users involved.
- Create shared groups and manage their members while preventing duplicate entries.
- Record income and expenses using a common transaction system with categories.
- Track how group expenses are divided among users and whether each person's share has been paid.
- Store repayments with details such as the payer, receiver, amount, payment method, and date.
- Maintain accurate and reliable records using database constraints and validation rules to prevent duplicate or inconsistent data.

3. Project Scope and Requirements

Functional area	Requirement
User and friend management	Manage user accounts, login methods, and friend requests, including accepting or blocking connections.
Group management	Create shared groups, choose group icons, and manage their members.
Expense and income recording	Record financial activities such as amount, date, category, and description.
Expense splitting	Divide group expenses equally or by custom amounts and track each member’s share.
Settlement tracking	Record repayments between users and keep track of pending dues.
History and categories	Organise transactions by category and retain useful details for reviewing past activity.

4. Project Modules

1. Identity and authentication

Primary tables: *User, AuthProvider, and Icon*
Store user details, profile icons, and linked login providers. Mobile numbers must be unique, while password hashes are handled by the authentication system.

2. Friendship management

Primary tables: *Friendship and User*
Maintain connections between users, including request status, who performed an action, and when the friendship was established. Rules are needed to handle acceptance and blocking consistently.

3. Group and membership management

Primary tables: *SplitGroup, SplitGroupMember, User, Icon*
Store group details, identify the creator, and manage members. The composite key in SplitGroupMember prevents the same user from being added to a group more than once.

4. Category and transaction management

Primary tables: *Category and Transaction*
Organize financial records by category and transaction type. The Transaction table supports income, group expenses, and direct loans, along with details such as amount, date, description, and receipt reference.

5. Expense-splitting engine

Primary tables: *Transaction and ExpenseSplit*
Generate an allocation row for each person who owes part of a transaction. Equal/custom splits must validate that the allocated amounts reconcile to the amount to be shared.

6. Settlement and balance reconciliation

Primary tables: *Settlement, ExpenseSplit, SplitGroupMember*
Record repayments between users and update outstanding obligations accordingly. Group balances should be recalculated consistently to reflect the recorded expenses and settlements.

7. History and reporting

Primary tables: *Transaction, Category, Settlement*
Use stored transaction details and repayment records to review past activity and prepare expense summaries. These reports would be presented through the application layer built on top of the database.

5. Existing Similar Projects

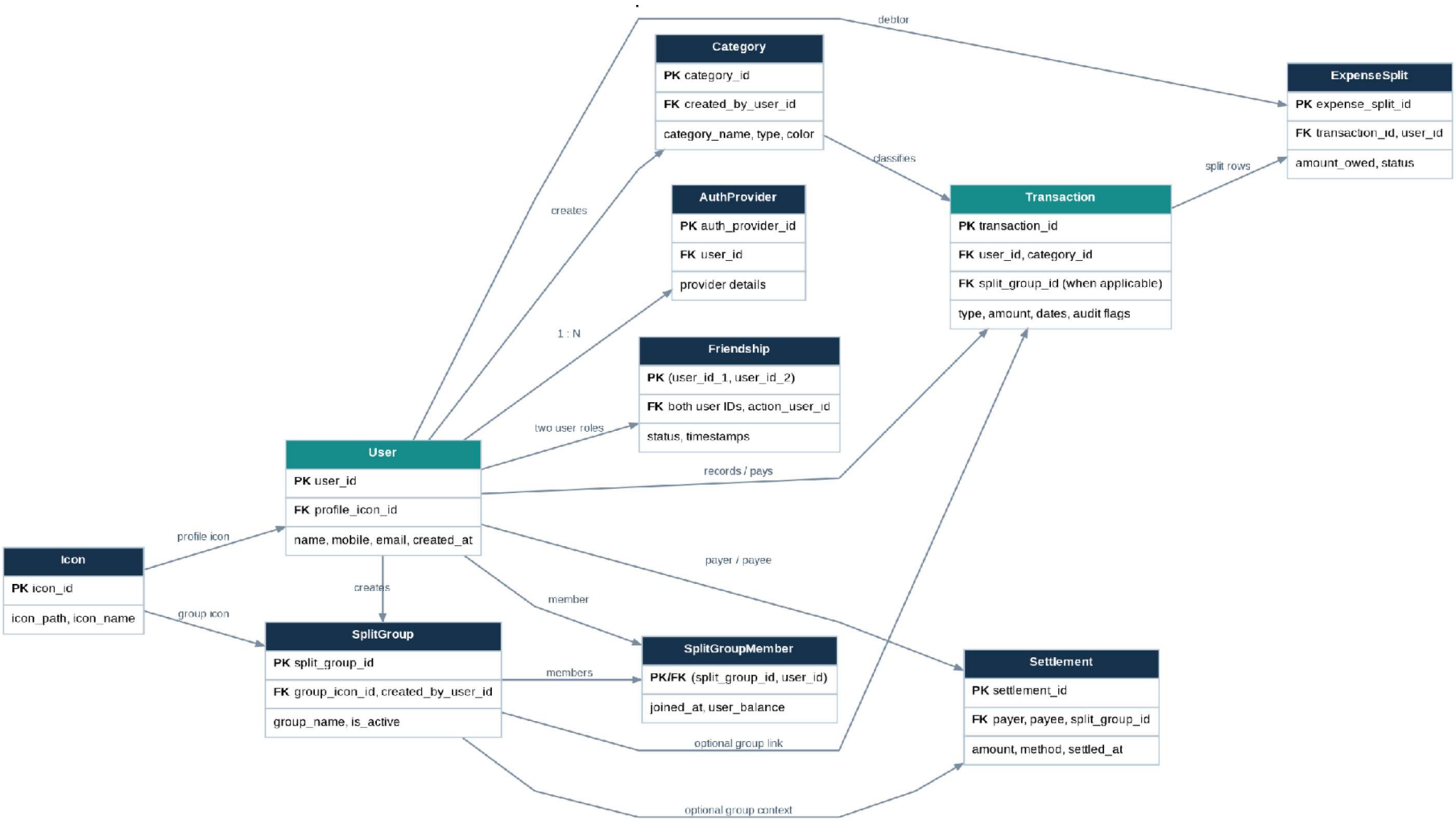
Existing solution	Described capabilities	Relevance to FinTrack
Splitwise	Tracks shared expenses, split bills,and manages balances and repayments.	A useful reference for dividing expenses and tracking who owes whom.
tricount	Supports group expenses, flexible splits, repayments, receipt photos, and multiple currencies.	Offers ideas for making group expense tracking simple and convenient.
Settle Up	Records shared expenses, synchronizes group records, and helps reduce the number of repayments needed.	Provides a reference for displaying balances and simplifying settlements.
Manual spreadsheets / chat notes	Allow flexible expense tracking but often require manual calculations and updates.	FinTrack's relational database provides a more structured way to connect expenses, users, and repayments.

6. Gaps and Proposed Novelty

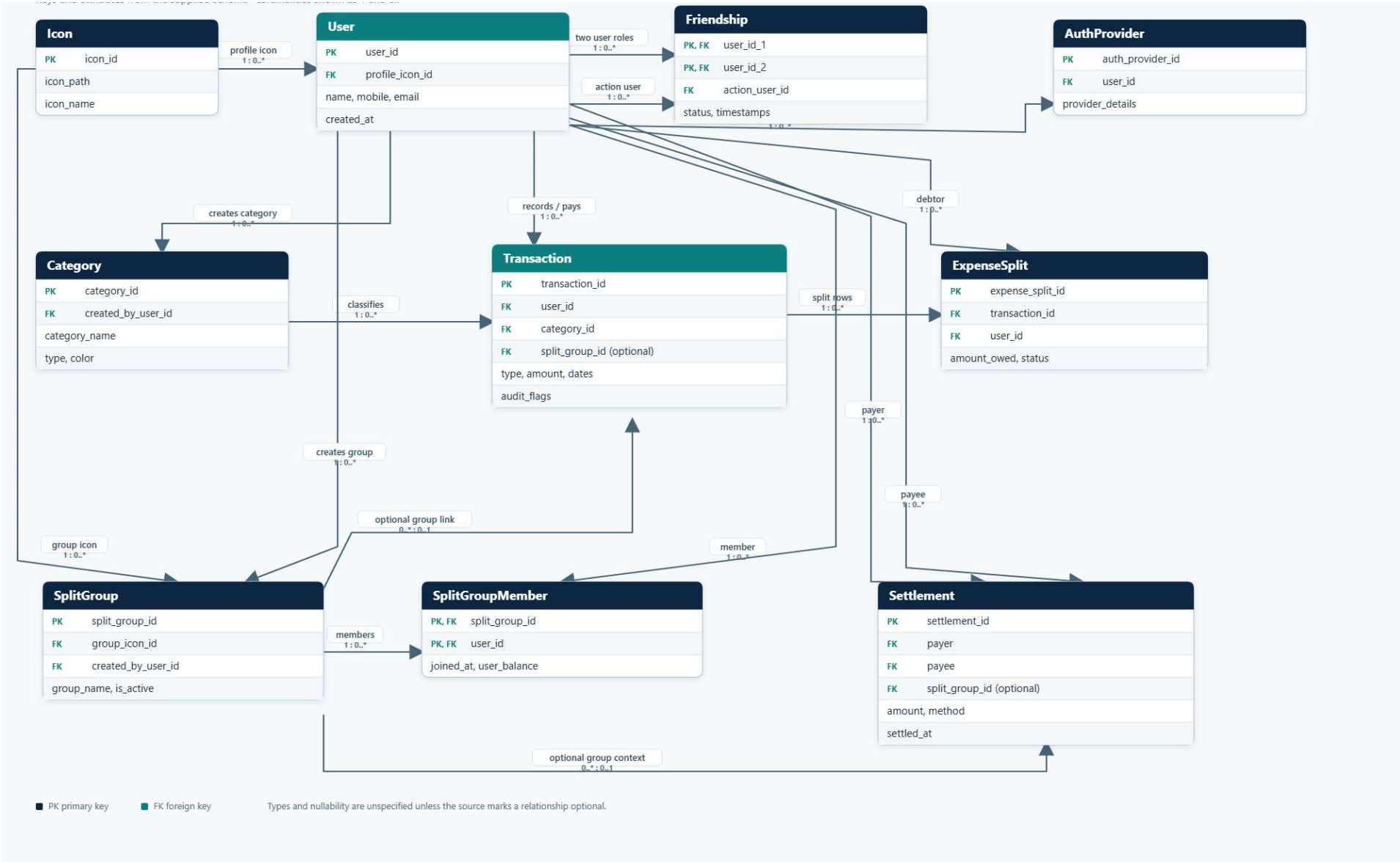
Gap / design opportunity	What the supplied schema already provides	Proposed novelty / value
A single clear financial event model	The detailed project schema uses Transaction plus transaction_type for INCOME, GROUP_SPLIT, and DIRECT_LOAN.	A simple, transparent single-table design keeps shared data consistent and clearly defines rules for each event type. It must be checked to ensure every type is linked to the correct group and split records.
Explicit lifecycle for friend relationships	Friendship stores a composite user pair, status, action_user_id, created_at, and established_at.	The project shows how relationships can be managed through structured data and defined stages instead of a simple contact list.
Group membership as a junction relation	SplitGroupMember uses the composite primary key (split_group_id, user_id).	The schema directly represents the many-to-many relationship and avoids duplicate membership pairs when the key is enforced.
Separate allocation records from the original transaction	ExpenseSplit references Transaction and User and records amount_owed and status.	Each person's debt is linked to the original transaction, making it easier to track payments and manage disputes or transaction history.
Settlement records with explicit payer and payee	Settlement stores payer_user_id, payee_user_id, amount, payment_method, transaction_reference, settled_at, and group context.	Repayments can be tracked separately from expenses, allowing the app to calculate remaining balances from recorded transactions and payments.
Identity and display assets separated from core entities	AuthProvider and Icon are separate relations.	Different login methods and reusable profile or group icons can be managed separately, avoiding duplicate data in the User and SplitGroup tables.
Audit-aware transaction storage	Transaction includes is_deleted, deleted_at, created_at, and updated_at.	Using consistent deletion rules and timestamps improves tracking, while ensuring deleted records are excluded from balance calculations when necessary.

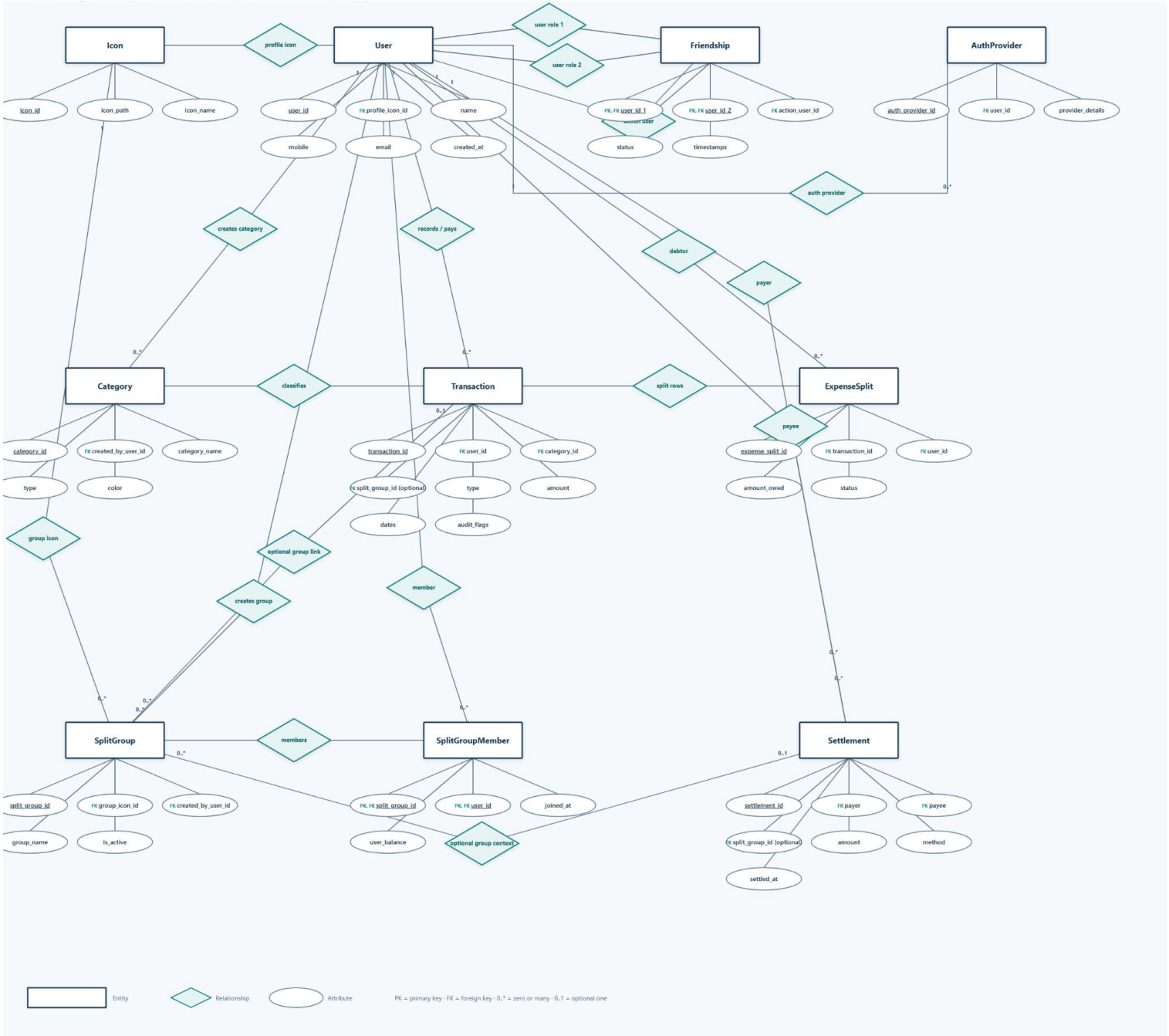
7. Schema Diagram

Figure 1. FinTrack relational schema overview. PK = primary key; FK = foreign key.



8. Entity-Relationship (ER) Diagram




Icon
profile icon
User
user role 1
user role 2
Friendship
AuthProvider
icon_id
icon_path
icon_name
user_id
FK profile_icon_id
name
mobile
email
created_at
PK, FK user_id 1
PK, FK user_id 2
FK action_user_id
status
timestamps
auth_provider_id
FK user_id
provider_details
auth provider
creates category
records / pays
debtor
payer
Category
classifies
Transaction
split rows
ExpenseSplit
payee
category_id
FK created_by_user_id
category_name
type
color
transaction_id
FK user_id
FK category_id
FK split_group_id (optional)
type
amount
dates
audit_flags
expense_split_id
FK transaction_id
FK user_id
amount_owed
status
group icon
optional group link
creates group
member
SplitGroup
members
SplitGroupMember
Settlement
split_group_id
FK group_icon_id
FK created_by_user_id
group_name
is_active
PK, FK split_group_id
PK, FK user_id
joined_at
user_balance
optional group context
settlement_id
FK payer
FK payee
FK split_group_id (optional)
amount
method
settled_at
Entity
Relationship
Attribute
PK = primary key · FK = foreign key · 0..* = zero or many · 0..1 = optional one

9. Relational Data Dictionary

9.1 Icon

Reusable vector assets displayed for user profiles and group icons.

Attribute	Key / constraint stated in source	Purpose
icon_id	PK, AI, NN	Unique identifier generated automatically.
icon_path	NN	Path or location of the icon asset.
icon_name	NN	Human-readable icon name.

9.2 User

Registered platform user and core contact/profile data.

Attribute	Key / constraint stated in source	Purpose
user_id	PK, AI, NN	Unique user identifier.
first_name	NN	User's first name.
middle_name	Nullable	Optional middle name.
last_name	Nullable	Optional last name.
mobile_number	NN, UQ	Required mobile number; unique across users.
email_id	Nullable	Optional email address.
profile_icon_id	FK -> Icon.icon_id	Optional profile icon reference; nullability is not further specified in the source.
created_at	Timestamp, NN	Time the user record was created.

9.3 AuthProvider

Authentication method associated with a user account.

Attribute	Key / constraint stated in source	Purpose
auth_provider_id	PK	Identifier for this authentication-provider record.
user_id	FK -> User.user_id	User to which the provider is linked.
provider_name	Not specified	Name of the provider, for example a local login or OAuth provider.
provider_user_id	Not specified	Identifier returned by the external provider, where applicable.
password_hash	Not specified	Password hash for a local credential, where applicable.
linked_at	Not specified	Time the provider was linked.
last_login	Not specified	Time of the last login using the provider.

9.4 Friendship

Bidirectional social link between two distinct users, with relationship state.

Attribute	Key / constraint stated in source	Purpose
user_id_1	Composite PK, FK -> User.user_id	First participant in the friendship pair.
user_id_2	Composite PK, FK -> User.user_id	Second participant; the two IDs together identify the row.
status	Not specified	Friendship/request state, such as pending, accepted, or blocked as defined by the implementation.
action_user_id	FK -> User.user_id	User who performed the latest relevant friendship action.
created_at	Not specified	Time the relationship/request record was created.
established_at	Not specified	Time the relationship became established, if applicable.

9.5 SplitGroup

Collaborative space for shared bills and expense tracking.

Attribute	Key / constraint stated in source	Purpose
split_group_id	PK	Unique group identifier.
group_name	Not specified	Display name of the group.
description	Not specified	Optional group description.
group_icon_id	FK -> Icon.icon_id	Icon associated with the group.
created_by_user_id	FK -> User.user_id	User who created the group.
created_at	Not specified	Time the group was created.
is_active	Not specified	Whether the group is active.

9.6 SplitGroupMember

Junction relation recording membership and an individual group balance.

Attribute	Key / constraint stated in source	Purpose
split_group_id	Composite PK, FK -> SplitGroup.split_group_id	Group in which the user participates.
user_id	Composite PK, FK -> User.user_id	Member user; paired with split_group_id as the composite key.
joined_at	Not specified	Time the user joined the group.
user_balance	Not specified	Member's net standing balance within that group.

9.7 Category

Classification tag for income and expense transactions.

Attribute	Key / constraint stated in source	Purpose
category_id	PK	Unique category identifier.
category_name	Not specified	Display name of the category.
category_type	Not specified	Type of category; permitted values are not specified in the source.

Attribute	Key / constraint stated in source	Purpose
category_color_hex	Not specified	Color value for display.
created_by_user_id	FK -> User.user_id	User who created the category.

9.8 Transaction

Financial event representing income, a group split, or a direct loan.

Attribute	Key / constraint stated in source	Purpose
transaction_id	PK	Unique financial-event identifier.
user_id	FK -> User.user_id	User associated with / recording the transaction.
transaction_type	Discriminator	Source lists INCOME, GROUP_SPLIT, and DIRECT_LOAN.
category_id	FK -> Category.category_id	Category assigned to the transaction.
split_group_id	FK -> SplitGroup.split_group_id; optional by type	Required for GROUP_SPLIT; specified as NULL for INCOME and DIRECT_LOAN in the source narrative.
amount	Not specified	Transaction amount.
description	Not specified	Human-readable description.
receipt_image_url	Not specified	Reference to an attached receipt image.
transaction_date	Not specified	Date of the financial event.
is_deleted	Not specified	Soft-delete flag.
deleted_at	Not specified	Deletion timestamp when soft deleted.
created_at	Not specified	Record creation timestamp.
updated_at	Not specified	Last update timestamp.

9.9 ExpenseSplit

Individual person’s allocation / debt obligation for a transaction.

Attribute	Key / constraint stated in source	Purpose
expense_split_id	PK	Unique allocation identifier.
transaction_id	FK -> Transaction.transaction_id	Transaction from which the obligation arises.
user_id	FK -> User.user_id	User who owes the allocated amount.
amount_owed	Not specified	Amount allocated to this user.
status	Not specified	State of the allocation, including whether it is settled.

9.10 Settlement

Recorded reimbursement between two users.

Attribute	Key / constraint stated in source	Purpose
settlement_id	PK	Unique repayment identifier.
payer_user_id	FK -> User.user_id	User who paid the reimbursement.

Attribute	Key / constraint stated in source	Purpose
payee_user_id	FK -> User.user_id	User who received the reimbursement.
split_group_id	FK -> SplitGroup.split_group_id	Group associated with the settlement, if any; nullability is not explicitly stated.
amount	Not specified	Reimbursement amount.
payment_method	Not specified	Method used for the repayment.
transaction_reference	Not specified	External or internal payment reference.
settled_at	Not specified	Time the settlement was recorded.

10. Conclusion

FinTrack aims to make managing shared expenses simpler and more organized by bringing users, groups, transactions, expense splits, and repayments into one relational database. Its design provides a structured way to track financial activity, understand who owes whom, and maintain a clear record of how outstanding balances are formed.

The project also highlights the importance of good database design. By using primary keys, foreign keys, composite keys, and appropriate constraints, FinTrack aims to maintain consistent records and reduce duplication and errors. Its modular structure allows different parts of the system, such as friendship management, group expenses, and settlements, to work together.

Overall, FinTrack provides a foundation for a shared-expense management application. The next step would be to implement the database operations and settlement logic, test different expense-splitting scenarios, and build a user-friendly interface around the system. With these additions, the project could become a practical solution for friends, roommates, and groups who want to manage shared finances with greater clarity and less manual effort.

11. References

[1] Relational database fundamentals. E. F. Codd, “A relational model of data for large shared data banks,” Communications of the ACM, vol. 13, no. 6, pp. 377–387, 1970. https://doi.org/10.1145/362384.362685

[2] Entity-Relationship modelling. P. P.-S. Chen, “The entity-relationship model—toward a unified view of data,” ACM Transactions on Database Systems, vol. 1, no. 1, pp. 9–36, 1976. https://doi.org/10.1145/320434.320440

[3] Algorithms for settling multiple debts. T. Verhoeff, “Settling multiple debts efficiently: An invitation to computing science,” Informatics in Education, vol. 3, no. 1, pp. 105–126, 2004. https://doi.org/10.15388/infedu.2004.08

[4] tricount. “Group expense tracking.” Describes recording group expenses and suggesting repayments from balances. https://www.tricount.com/en-in/features/group-expense-tracking

[5] Fair cost-sharing mechanisms. C.-K. Chau and K. Elbassioni, “Quantifying inefficiency of fair cost-sharing mechanisms for sharing economy,” IEEE Transactions on Control of Network Systems, vol. 5, no. 4, pp. 1809–1818, 2018. https://doi.org/10.1109/TCNS.2017.2763747

[6] Translating ER models into relational schemas. V. C. Storey, “Relational database design based on the entity-relationship model,” Data & Knowledge Engineering, vol. 7, no. 1, pp. 47–83, 1991. https://doi.org/10.1016/0169-023X(91)90033-T